MikroTik RouterOS Log Integration

with Graylog & Wazuh SIEM

Complete Step-by-Step Configuration Guide

Internship Technical Annex — Version 2.0

Architecture Overview

`MikroTik Routers` $CEF + TLS$ `Graylog Server` $rsyslog(localhost)$ `Wazuh Agent` $AES-256$ `Wazuh Manager`

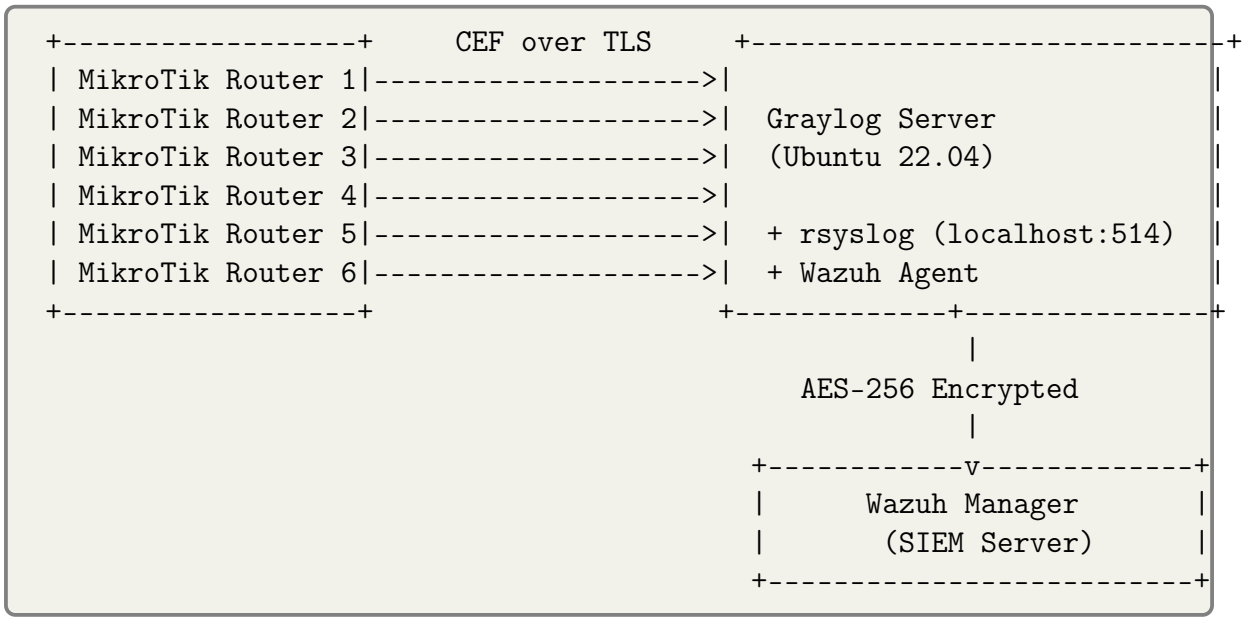
April 2026

Contents

Introduction

This guide covers the complete configuration of a centralized, secure log management system using MikroTik RouterOS, Graylog, and Wazuh SIEM. The goal is to collect security logs from multiple MikroTik routers, forward them securely to a Graylog server, and then analyze them with Wazuh for threat detection and alerting.

Architecture Summary



Log Flow Explanation

1. MikroTik routers send logs in **CEF format over TLS** to Graylog on port 6514
2. Graylog receives, parses, and stores logs in its **OpenSearch** backend (for visualization)
3. Graylog forwards logs via a **Syslog Output plugin** to rsyslog on localhost:514
4. rsyslog writes the logs to **/var/log/mikrotik.log**
5. The **Wazuh Agent** reads this file and forwards events to Wazuh Manager encrypted with AES-256
6. Wazuh applies **custom decoders and rules** for threat detection

Note: The localhost channel between Graylog and rsyslog uses TCP on port 514. Since this communication stays on the same machine, it is never exposed to the network and cannot be sniffed.

Prerequisites

- MikroTik CHR running RouterOS 7.18 or later (TLS for CEF logging requires 7.18+)
- Ubuntu 22.04 server for Graylog (minimum 8GB RAM)
- Wazuh Manager server (version 4.x or later)
- Network connectivity between all components
- Administrative access to all systems

Part 1: Graylog Server Installation (Ubuntu 22.04)

Step 1: Set Server Timezone

Listing 1: Set timezone to UTC

```
sudo timedatectl set-timezone UTC
```

Step 2: Install MongoDB

MongoDB serves as the metadata database for Graylog.

Listing 2: Install MongoDB 8.0

```
# Install required tools
sudo apt-get install gnupg curl

# Import MongoDB public key
curl -fsSL https://www.mongodb.org/static/pgp/server-8.0.asc | \
   sudo gpg -o /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor

# Add MongoDB repository (Ubuntu 22.04 - Jammy)
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/\
mongodb-server-8.0.gpg ] https://repo.mongodb.org/apt/ubuntu \
jammy/mongodb-org/8.0 multiverse" | \
sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list

# Update and install
sudo apt-get update
sudo apt-get install -y mongodb-org

# Prevent automatic upgrades
sudo apt-mark hold mongodb-org
```

Listing 3: Configure MongoDB to listen on all interfaces

```
sudo nano /etc/mongod.conf
# Add/modify the following:
# net:
```

```
#   port: 27017
#   bindIpAll: true
```

Listing 4: Enable and start MongoDB

```
sudo systemctl daemon-reload
sudo systemctl enable mongod.service
sudo systemctl start mongod.service
sudo systemctl status mongod.service
```

Step 3: Install Graylog Data Node

Listing 5: Install Graylog Data Node

```
# Download and install Graylog repository
wget https://packages.graylog2.org/repo/packages/graylog-7.0-
   repository_latest.deb
sudo dpkg -i graylog-7.0-repository_latest.deb
sudo apt-get update
sudo apt-get install graylog-datanode

# Check and set vm.max_map_count
cat /proc/sys/vm/max_map_count
echo 'vm.max_map_count=262144' | \
sudo tee -a /etc/sysctl.d/99-graylog-datanode.conf
sudo sysctl --system
```

Listing 6: Generate password secret (save this value!)

```
openssl rand -hex 32
# IMPORTANT: Save this value - needed for both Data Node and
   Graylog
```

Listing 7: Configure Data Node

```
sudo nano /etc/graylog/datanode/datanode.conf

# Add/modify these values:
# password_secret = <YOUR_GENERATED_SECRET>
# opensearch_heap = 4g        # Half of your total RAM
# mongodb_uri = mongodb://localhost:27017/graylog
```

Listing 8: Start Data Node

```
sudo systemctl daemon-reload
sudo systemctl enable graylog-datanode.service
sudo systemctl start graylog-datanode
sudo systemctl status graylog-datanode
```

Step 4: Install Graylog Server

Listing 9: Install Graylog Open

```
sudo apt-get install graylog-server
```

Listing 10: Generate admin password hash

```
echo -n "Enter Password: " && head -1 </dev/stdin | \
tr -d '\n' | sha256sum | cut -d" " -f1
# Save both the plain password and the hash
```

Listing 11: Configure Graylog server

```
sudo nano /etc/graylog/server/server.conf

# Set these values:
# password_secret = <SAME_SECRET_AS_DATANODE>
# root_password_sha2 = <YOUR_PASSWORD_HASH>
# http_bind_address = 0.0.0.0:9000
# message_journal_max_age = 12h
# message_journal_max_size = 5gb
```

Listing 12: Configure heap settings

```
sudo nano /etc/default/graylog-server

# Set:
# GRAYLOG_SERVER_JAVA_OPTS="-Xms2g -Xmx2g -server \
#   -XX:+UseG1GC -XX:-OmitStackTraceInFastThrow"
```

Listing 13: Start Graylog

```
sudo systemctl daemon-reload
sudo systemctl enable graylog-server.service
sudo systemctl start graylog-server.service
sudo systemctl status graylog-server.service
```

Note: Access Graylog web interface at `http://YOUR_SERVER_IP:9000`. On first login, use the preflight credentials found in the log file: `sudo journalctl -u graylog-server | grep -i "password"`

Part 2: TLS Certificate Generation

Step 5: Generate TLS Certificate for Graylog

Listing 14: Create certificate directory and generate certificate

```
sudo mkdir -p /etc/graylog/server/certs
cd /etc/graylog/server/certs

```

```
# Generate certificate with TLS server key usage
# CRITICAL: extendedKeyUsage=serverAuth is required for MikroTik TLS
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout /etc/graylog/server/certs/graylog.key \
    -out /etc/graylog/server/certs/graylog.crt \
    -subj "/CN=graylog" \
    -addext "extendedKeyUsage=serverAuth" \
    -addext "keyUsage=digitalSignature,keyEncipherment"

# Set correct permissions
sudo chown graylog:graylog /etc/graylog/server/certs/graylog.key
sudo chown graylog:graylog /etc/graylog/server/certs/graylog.crt
sudo chmod 600 /etc/graylog/server/certs/graylog.key

# Restart Graylog to apply new certificate
sudo systemctl restart graylog-server
```

Warning: The certificate MUST include `extendedKeyUsage=serverAuth` and `keyUsage=digitalSignature,keyEncipherment`. Without these extensions, MikroTik RouterOS will NOT display TLS as an available protocol option, and the connection will fail silently.

Part 3: Graylog Input Configuration

Step 6: Create CEF TCP Input with TLS

1. Open Graylog web interface: `http://YOUR_IP:9000`
2. Navigate to **System -> Inputs**
3. Select **CEF TCP** from the dropdown
4. Click **Launch new input**
5. Configure the following fields:

Field	Value
Title	MikroTik CEF TLS
Bind address	0.0.0.0
Port	6514
Timezone	UTC
TLS cert file	/etc/graylog/server/certs/graylog.crt
TLS private key file	/etc/graylog/server/certs/graylog.key
Enable TLS	checked

Note: Port 6514 is used instead of 514 because ports below 1024 require root privileges on Linux. Port 6514 is the standard port for syslog over TLS.

Listing 15: Verify Graylog is listening on port 6514

```
sudo ss -tulnp | grep 6514
# Expected: tcp LISTEN 0 4096 *:6514 *:* users:(("java",...))
```

Part 4: MikroTik RouterOS Configuration

Step 7: Transfer Certificate to MikroTik

Listing 16: Transfer certificate from Graylog server to MikroTik

```
# Run from the Graylog server for each router
scp /etc/graylog/server/certs/graylog.crt admin@ROUTER_IP:graylog.
    crt
```

Step 8: Import Certificate on MikroTik

Listing 17: Import and verify certificate on MikroTik

```
# Import the Graylog certificate
/certificate import file-name=graylog.crt passphrase=""

# Verify the certificate has correct key usage
/certificate print detail
# Must show: key-usage=digital-signature,key-encipherment,tls-
    server
# Must show: trusted=yes
```

Warning: If the certificate does not show `key-usage=tls-server`, the TLS protocol option will NOT appear in the logging action configuration. Regenerate the certificate with the correct extensions.

Step 9: Configure Remote Logging Action

Listing 18: Create logging action with CEF over TLS

```
# CRITICAL: remote-protocol=tls MUST be set in the add command
# It CANNOT be changed afterwards with the set command (syntax
    error)
/system logging action add \
    name=graylogcef \
    target=remote \
    remote=GRAYLOG_SERVER_IP \
    remote-port=6514 \
    remote-log-format=cef \
```

```
    remote-protocol=tls

# Verify
/system logging action print detail
# Expected: remote-protocol=tls
```

Warning: Critical Discovery: The `remote-protocol=tls` parameter MUST be included in the initial `add` command. RouterOS will return a syntax error if you try to set it afterwards. This is a known behavior in RouterOS 7.x with CEF format.

Step 10: Configure Logging Topics

Listing 19: Add logging rules for Router 1

```
/system logging add topics=firewall prefix="MKT[001][FW]" action=
   graylogcef
/system logging add topics=system prefix="MKT[001][SYS]" action=
   graylogcef
/system logging add topics=info prefix="MKT[001][INFO]" action=
   graylogcef
```

Note: The prefix format `MKT[ID][TYPE]` serves two purposes: the ID (001, 002...) uniquely identifies each router, and the type (FW, SYS, INFO) categorizes the log. Both are extracted by Wazuh decoders for structured alerting.

Step 11: Configure Additional Routers

Repeat Steps 7-10 for each router, incrementing the router ID:

Listing 20: Router 2 example

```
/certificate import file-name=graylog.crt passphrase=""

/system logging action add \
    name=graylogcef \
    target=remote \
    remote=GRAYLOG_SERVER_IP \
    remote-port=6514 \
    remote-log-format=cef \
    remote-protocol=tls

/system logging add topics=firewall prefix="MKT[002][FW]" action=
   graylogcef
/system logging add topics=system prefix="MKT[002][SYS]" action=
   graylogcef
/system logging add topics=info prefix="MKT[002][INFO]" action=
   graylogcef
```

Part 5: Graylog Syslog Output Plugin

This section covers forwarding logs from Graylog to the local Wazuh Agent via rsyslog.

Step 12: Install Syslog Output Plugin

The Graylog open-source version only includes GELF and STDOUT outputs natively. To forward logs in syslog format, we use a community plugin.

Listing 21: Download and install the syslog output plugin

```
# Download the plugin JAR
wget https://github.com/wizecore/graylog2-output-syslog/releases/\
latest/download/graylog-output-syslog-6.3.5.jar

# Install into Graylog plugins directory
sudo cp graylog-output-syslog-6.3.5.jar /usr/share/graylog-server/
   plugin/

# Restart Graylog to load the plugin
sudo systemctl restart graylog-server
```

Note: This plugin was developed for Graylog 6.3.5+ but works on Graylog 7.x. After restarting, a new "Syslog" output type will appear in System -> Outputs with TCP, UDP, and SSL/TLS options.

Step 13: Configure rsyslog to Receive from Graylog

rsyslog is already installed on Ubuntu by default. Configure it to listen on TCP port 514 and write to a dedicated log file.

Listing 22: Configure rsyslog

```
sudo nano /etc/rsyslog.d/graylog-wazuh.conf
```

Add the following content:

Listing 23: rsyslog configuration for Graylog forwarding

```
# Load TCP input module
module(load="imtcp")
input(type="imtcp" port="514")

# Write logs received from localhost to mikrotik log file
if $fromhost-ip == '127.0.0.1' then /var/log/mikrotik.log
& stop
```

Listing 24: Restart rsyslog and verify

```
sudo systemctl restart rsyslog

# Verify rsyslog is listening on port 514
sudo ss -tulnp | grep 514
# Expected: tcp LISTEN 0 25 0.0.0.0:514 ...  rsyslogd
```

Step 14: Create Graylog Syslog Output

1. Navigate to **System -> Outputs**
2. Select **Syslog Output** from the dropdown
3. Click **Launch new output**
4. Configure:

Field	Value
Title	Wazuh Agent Local
Protocol	TCP
Remote host	127.0.0.1
Syslog port	514
Message format	full

Note: Use **full** format (not plain) to preserve all CEF fields including the `msg: MKT[001][SYS]:...` field that Wazuh decoders use for parsing.

Step 15: Assign Output to Stream

1. Navigate to **Streams**
2. Find **Default Stream**
3. Click **More Actions -> Manage Outputs**
4. Select the **Wazuh Agent Local** output

Warning: An output will NOT forward messages unless it is assigned to a stream. The Default Stream captures all messages, making it the simplest option for this lab setup.

Part 6: Wazuh Agent Installation and Configuration

Step 16: Install Wazuh Agent

Listing 25: Add Wazuh repository and install agent

```
# Add Wazuh GPG key
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | \
sudo gpg --no-default-keyring \
--keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg \
--import && sudo chmod 644 /usr/share/keyrings/wazuh.gpg

# Add Wazuh repository
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
```

```
https://packages.wazuh.com/4.x/apt/ stable main" | \
sudo tee /etc/apt/sources.list.d/wazuh.list

# Install agent
sudo apt-get update
sudo WAZUH_MANAGER="WAZUH_MANAGER_IP" apt-get install wazuh-agent

# Start agent
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

Step 17: Configure Wazuh Agent to Read MikroTik Logs

Listing 26: Edit Wazuh agent configuration

```
sudo nano /var/ossec/etc/ossec.conf
```

Add this block before `</ossec_config>`:

Listing 27: localfile block for mikrotik.log

```
<localfile>
    <log_format>syslog</log_format>
    <location>/var/log/mikrotik.log</location>
</localfile>
```

Listing 28: Restart agent to apply changes

```
sudo systemctl restart wazuh-agent
```

Part 7: Wazuh Custom Decoders

The logs forwarded by Graylog use the `full` format which wraps the original MikroTik message. Custom decoders are needed to extract structured fields.

Step 18: Add MikroTik Graylog Decoders

On the **Wazuh Manager** server:

Listing 29: Edit decoders file on Wazuh Manager

```
sudo nano /var/ossec/etc/decoders/local_decoder.xml
```

Add the following decoders:

Listing 30: MikroTik Graylog decoders

```
<!-- MikroTik via Graylog Full Format - Parent Decoder -->
<decoder name="mikrotik_graylog">
    <prematch>msg: MKT[</prematch>
    <type>syslog</type>
</decoder>
```

```
6
7  <!-- Authentication Failure -->
8  <decoder name="mikrotik_graylog_auth_failure">
9      <parent>mikrotik_graylog</parent>
10     <use_own_name>true</use_own_name>
11     <prematch>login failure</prematch>
12     <regex type="pcre2">msg: MKT\[(\d+)\]\[([A-Z]+)\]:
13         (login failure) for user (\S+) from (\S+) via (\S+)</regex>
14     <order>router_id,log_type,act,user,srcip,access_method</order>
15 </decoder>
16
17 <!-- Authentication Success -->
18 <decoder name="mikrotik_graylog_auth_login">
19     <parent>mikrotik_graylog</parent>
20     <use_own_name>true</use_own_name>
21     <prematch>logged in</prematch>
22     <regex type="pcre2">msg: MKT\[(\d+)\]\[([A-Z]+)\]:
23         user (\S+) (logged in) from (\S+) via (\S+)</regex>
24     <order>router_id,log_type,user,act,srcip,access_method</order>
25 </decoder>
```

Listing 31: Restart Wazuh Manager to apply decoders

```
1 sudo systemctl restart wazuh-manager
```

Step 19: Verify Decoder with wazuh-logtest

Listing 32: Test decoder with a sample log

```
1 sudo /var/ossec/bin/wazuh-logtest
```

Paste this test log:

```
1 Apr 30 08:45:07 ubuntu22-desktop unknown source: 172.30.2.38 |
2 message: CHR: [10, High] { msg: MKT[001][SYS]: login failure
3 for user admin from 172.30.3.71 via winbox | device_vendor:
     MikroTik }
```

Expected output:

Listing 33: Expected decoder output

```
1 **Phase 2: Completed decoding.
2     name: 'mikrotik_graylog_auth_failure'
3     parent: 'mikrotik_graylog'
4     router_id: '001'
5     log_type: 'SYS'
6     act: 'login failure'
7     user: 'admin'
8     srcip: '172.30.3.71'
9     access_method: 'winbox'
```

Part 8: Verification

Step 20: Verify the Complete Pipeline

Listing 34: Check all services are running

```
# On Graylog server
sudo systemctl status graylog-server
sudo systemctl status graylog-datanode
sudo systemctl status mongod
sudo systemctl status rsyslog
sudo systemctl status wazuh-agent

# Check ports
sudo ss -tulnp | grep -E "514|6514|9000"
```

Listing 35: Monitor log file in real time

```
sudo tail -f /var/log/mikrotik.log | grep MKT
```

Listing 36: Generate test logs from MikroTik

```
# Attempt login with wrong credentials to generate auth failure log
# The log should appear in Graylog Search and Wazuh Dashboard
   within seconds
```

Step 21: Troubleshooting

Issue	Cause	Solution
TLS not available in RouterOS	Missing tls-server in cert	Regenerate with extendedKeyUsage=serverAuth
syntax error on remote-protocol=tls	Used set instead of add	Delete action and recreate with add
No logs in mikrotik.log	Graylog output not assigned	Assign output to Default Stream
Logs in file but not in Wazuh	Wrong localfile path	Check ossec.conf localfile location
Decoder not matching	Wrong prematch	Use wazuh-logtest to debug
Graylog fails to start	Journal too large	Set message_journal_max_size = 5gb
Plugin output format wrong	Using plain format	Switch output format to full

Part 9: Active Response - Automated Email Alert

This section covers the configuration of an automated email alert system that triggers when a brute force attack is detected on any MikroTik router.

Step 22: Configure Mail on the Wazuh Agent

Listing 37: Install mail utilities on the Graylog/Agent server

```
# Install mailutils and postfix
sudo apt install mailutils -y
# Choose "Internet Site" during Postfix configuration
```

Listing 38: Install jq for JSON parsing

```
# jq is required by the active response script
sudo apt install jq -y
```

Step 23: Create the Active Response Script

Listing 39: Create the custom mail alert script

```
sudo nano /var/ossec/active-response/bin/custom_wazuh_mail.sh
```

Listing 40: Active response script content

```
#!/bin/bash
# ============================================================
# Wazuh Active Response - Custom Mail Alert for MikroTik
# Compatible with Wazuh 4.x stateful active response format
# ============================================================

LOG_FILE="/var/ossec/logs/active-responses.log"
MAIL_TO="admin1@domain.com,admin2@domain.com"

# READ INPUT (Wazuh 4.x format)
# Use timeout to prevent deadlock when reading stdin
INPUT=$(timeout 3 cat)

echo "$(date '+%Y-%m-%d %H:%M:%S') - [AR] Script triggered" >> "$LOG_FILE"

# Validate JSON input
if ! echo "$INPUT" | jq -e . > /dev/null 2>&1; then
    echo "$(date '+%Y-%m-%d %H:%M:%S') - [AR] ERROR: Invalid JSON" >> "$LOG_FILE"
    exit 1
fi

# Parse command and alert
COMMAND=$(echo "$INPUT" | jq -r '.command // empty')
ALERT=$(echo "$INPUT" | jq -c '.parameters.alert // empty')

echo "$(date '+%Y-%m-%d %H:%M:%S') - [AR] Command: $COMMAND" >> "$LOG_FILE"

if [ -z "$COMMAND" ] || [ -z "$ALERT" ]; then
```

```
    echo "$(date '+%Y-%m-%d %H:%M:%S') - [AR] ERROR: Missing data" >> "$LOG_FILE"
    exit 1
fi

if [ "$COMMAND" = "add" ]; then
    # Extract alert fields
    RULE_ID=$(echo "$ALERT"   | jq -r '.rule.id          // "unknown"')
    RULE_DESC=$(echo "$ALERT" | jq -r '.rule.description // "unknown"')
    LEVEL=$(echo "$ALERT"     | jq -r '.rule.level       // "0"')
    SRCIP=$(echo "$ALERT"     | jq -r '.data.srcip       // "N/A"')
    DSTUSER=$(echo "$ALERT"   | jq -r '.data.dstuser     // "N/A"')
    ROUTER_ID=$(echo "$ALERT" | jq -r '.data.router_id   // "Unknown"')
    LOG_TYPE=$(echo "$ALERT"  | jq -r '.data.log_type    // "N/A"')
    METHOD=$(echo "$ALERT"    | jq -r '.data.access_method // "N/A"')
    AGENT=$(echo "$ALERT"     | jq -r '.agent.name       // "unknown"')
    AGENT_IP=$(echo "$ALERT"  | jq -r '.agent.ip         // "unknown"')
    TIMESTAMP=$(echo "$ALERT" | jq -r '.timestamp        // "N/A"')
    FIRED=$(echo "$ALERT"     | jq -r '.rule.firedtimes  // "1"')
    MITRE_ID=$(echo "$ALERT"  | jq -r '.rule.mitre.id    // "N/A"')
    MITRE_TACTIC=$(echo "$ALERT" | jq -r '.rule.mitre.tactic // "N/A"')
    FULL_LOG=$(echo "$ALERT"  | jq -r '.full_log // "no log"' | head -c 800)

    SUBJECT="[WAZUH] BRUTE FORCE MikroTik - Level ${LEVEL} \
- Router MKT[${ROUTER_ID}] - IP ${SRCIP}"

    BODY="
============================================================
   WAZUH SECURITY ALERT - MIKROTIK BRUTE FORCE
============================================================
Rule ID        : ${RULE_ID}
Description    : ${RULE_DESC}
Severity Level : ${LEVEL}/15
Fired Times    : ${FIRED}
Timestamp      : ${TIMESTAMP}

ATTACK INFORMATION
------------------
Router ID      : MKT[${ROUTER_ID}]
Log Type       : ${LOG_TYPE}
Source IP      : ${SRCIP}
Target User    : ${DSTUSER}
Access Method  : ${METHOD}
```

```
71
72 WAZUH AGENT
73 -----------
74 Agent Name      : ${AGENT}
75 Agent IP        : ${AGENT_IP}
76
77 MITRE ATT&CK    : ${MITRE_ID} (${MITRE_TACTIC})
78
79 RECOMMENDED ACTIONS
80 --------------------
81 [1] Block IP : iptables -A INPUT -s ${SRCIP} -j DROP
82 [2] Check router logs : MikroTik -> Log -> Filter ${SRCIP}
83 [3] Verify user account : ${DSTUSER}
84
85 FULL LOG (extract)
86 ------------------
87 ${FULL_LOG}
88 ============================================================"
89
90     if echo "$BODY" | mail -s "$SUBJECT" "$MAIL_TO" 2>> "$LOG_FILE
    "; then
91         echo "$(date '+%Y-%m-%d %H:%M:%S') - [AR] MAIL SENT \
92 - Rule ${RULE_ID} - Router MKT[${ROUTER_ID}] \
93 - IP ${SRCIP} - User ${DSTUSER}" >> "$LOG_FILE"
94     else
95         echo "$(date '+%Y-%m-%d %H:%M:%S') - [AR] ERROR: Mail
    failed" \
96 >> "$LOG_FILE"
97         exit 1
98     fi
99 fi
100
101 if [ "$COMMAND" = "delete" ]; then
102     echo "$(date '+%Y-%m-%d %H:%M:%S') - [AR] DELETE action" >> "
    $LOG_FILE"
103 fi
104
105 exit 0
```

Listing 41: Set correct permissions on the script

```
1 sudo chmod 750 /var/ossec/active-response/bin/custom_wazuh_mail.sh
2 sudo chown root:wazuh /var/ossec/active-response/bin/
    custom_wazuh_mail.sh
```

Warning: **Critical:** Use `INPUT=$(timeout 3 cat)` instead of `INPUT=$(cat)` or `while read`. Using plain `cat` causes a deadlock because the script waits indefinitely for stdin to close. The `timeout 3` ensures the script continues after 3 seconds.

Warning: jq must be installed on the agent: The script uses `jq` for JSON parsing. If `jq` is not installed, all fields will fail to parse and the script will exit with "Invalid JSON input" error.

Step 24: Register the Command in Wazuh Manager

On the **Wazuh Manager** server, add the command and active response configuration:

Listing 42: Edit ossec.conf on Wazuh Manager

```
sudo nano /var/ossec/etc/ossec.conf
```

Add inside `<ossec_config>`:

Listing 43: Command and active response configuration

```
<!-- Register the custom mail command -->
<command>
    <name>custom_wazuh_mail</name>
    <executable>custom_wazuh_mail.sh</executable>
    <timeout_allowed>no</timeout_allowed>
</command>

<!-- Trigger on brute force rule 110015 -->
<active-response>
    <command>custom_wazuh_mail</command>
    <location>local</location>
    <rules_id>110015</rules_id>
</active-response>
```

Note: location:local means the script executes on the agent that generated the alert (ubuntu22-desktop / Graylog server). The script must be present on this agent, not only on the Wazuh Manager.

Listing 44: Restart Wazuh Manager and Agent

```
# On Wazuh Manager
sudo systemctl restart wazuh-manager

# On Graylog/Agent server
sudo systemctl restart wazuh-agent
```

Step 25: Verify Active Response

Listing 45: Verify the command is registered

```
# On Wazuh Manager
sudo /var/ossec/bin/agent_control -L
# Expected: Response name: custom_wazuh_mail0, command:
   custom_wazuh_mail.sh
```

Listing 46: Monitor active response logs on the agent

```
sudo tail -f /var/ossec/logs/active-responses.log
```

Expected output after a brute force attack:

```
2026-05-07 07:48:41 - [AR] Script triggered
2026-05-07 07:48:41 - [AR] Command: add
2026-05-07 07:48:41 - [AR] MAIL SENT - Rule 110015 - Router MKT [004] \
- IP 172.30.3.71 - User admin
```

Step 26: Add Mattermost Webhook Notification

Mattermost is a self-hosted messaging platform. Adding a webhook notification allows the security team to receive instant alerts in a dedicated channel.

Listing 47: Test Mattermost webhook connectivity

```
# Test from the agent server
# Note: Use internal IP and port 8065 (HTTP) for internal network access
# The public HTTPS URL requires nginx to allow internal connections
curl http://MATTERMOST_INTERNAL_IP:8065/hooks/YOUR_WEBHOOK_ID \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"text":"Test from Wazuh"}'
```

Note: HTTP vs HTTPS for Mattermost: Mattermost typically runs on HTTP port 8065 internally. The HTTPS URL (via nginx reverse proxy) may only be accessible from outside the network. From internal servers, use the direct HTTP connection on port 8065. When the nginx configuration is updated to allow internal IPs, you can switch to HTTPS.

Add the following to the active response script, after the mail sending block:

Listing 48: Mattermost webhook block in the script

```
# Add at the top of the script with other variables
MATTERMOST_WEBHOOK="http://MATTERMOST_IP:8065/hooks/YOUR_WEBHOOK_ID"

# Add after the mail block, inside if [ "$COMMAND" = "add" ]
# Send Mattermost alert
MM_TEXT="WAZUH BRUTE FORCE ALERT\nLevel: ${LEVEL}\nRouter: MKT[${ROUTER_ID}]\nSource IP: ${SRCIP}\nUser: ${DSTUSER}\nRule: ${RULE_DESC}\nTime: ${TIMESTAMP}"

curl -s -X POST \
  -H "Content-Type: application/json" \
  -d "{\"text\":\"$MM_TEXT\"}" \
  "$MATTERMOST_WEBHOOK" >> "$LOG_FILE" 2>&1

```

```
13 echo "$(date '+%Y-%m-%d %H:%M:%S') - [AR] Mattermost webhook sent"
   >> "$LOG_FILE"
```

Warning: Mail and Mattermost must be **independent blocks** — do not put Mattermost inside the mail `if` block. If mail fails, Mattermost should still send the alert.

Email Alert Example

```
Subject: [WAZUH] BRUTE FORCE MikroTik - Level 12 - Router MKT[004] - IP 172.30.3.71

Rule ID        : 110015
Description    : MikroTik: Possible brute force attack from 172.30.3.71
Severity Level : 12/15
Router ID      : MKT[004]
Source IP      : 172.30.3.71
Target User    : admin
Access Method  : winbox

RECOMMENDED ACTIONS
[1] Block IP : iptables -A INPUT -s 172.30.3.71 -j DROP
[2] Check router logs : MikroTik -> Log -> Filter 172.30.3.71
```

Summary

Final Architecture - Complete Pipeline

1. **MikroTik Routers** generate security logs (firewall, system, info)
2. Logs sent in **CEF format over TLS** to Graylog port 6514
3. **Graylog** parses CEF fields and stores in OpenSearch (visualization)
4. Graylog **Syslog Output plugin** forwards to rsyslog on localhost:514 (TCP)
5. **rsyslog** writes logs to /var/log/mikrotik.log
6. **Wazuh Agent** reads the file and forwards to Wazuh Manager (AES-256)
7. **Custom decoders** extract: router_id, log_type, srcip, user, action
8. **Wazuh rules** generate structured alerts with full context

Key Technical Discoveries

- **TLS requires correct certificate:** The Graylog certificate must include `extendedKeyUsage=serverAuth`. Without it, RouterOS will not show TLS as an option.
- **TLS must be set at creation:** `remote-protocol=tls` must be in the `add` command. Using `set` afterwards causes a syntax error.
- **Graylog output format matters:** Use `full` format (not `plain`) to preserve the `msg: MKT[...]` field needed by Wazuh decoders.
- **Localhost channel is secure:** TCP on 127.0.0.1 between Graylog and rsyslog never leaves the machine and cannot be intercepted.
- **Two copies of logs:** Graylog stores one copy in OpenSearch (for visualization) and forwards another copy to Wazuh (for security analysis).